

Database Management – SQL & PL/SQL

Total Time: 3 Hours

Theme: Personal Expense Tracker DB

Section A: Concept Application

1. Explain how relational databases help maintain accuracy in expense records.
2. Why are constraints important in personal finance data?
3. How does GROUP BY help analyze spending patterns?
4. Explain a scenario where rollback is required during expense entry.
5. How do views help users track monthly expenses efficiently?
6. Why use triggers for automatic category or balance updates?

Section B: SQL Hands-On

Given Database Schema (DO NOT MODIFY)

```
users (  
    user_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    email VARCHAR(100),  
    created_at DATE  
);
```

```
categories (  
    category_id INT PRIMARY KEY,  
    category_name VARCHAR(50)  
);
```

```
expenses (  
    expense_id INT PRIMARY KEY,  
    user_id INT,  
    category_id INT,  
    amount DECIMAL(10,2),  
    expense_date DATE,  
    FOREIGN KEY (user_id) REFERENCES users(user_id),  
    FOREIGN KEY (category_id) REFERENCES categories(category_id)  
);
```

1. DDL Understanding

Without modifying the schema:

- **Explain why foreign keys are used:** Specifically, why must `user_id` in the `expenses` table correspond to a real ID in the `users` table?
- **Mention one issue that would occur if foreign keys were removed:**
Describe the risk of "Orphaned Records" (e.g., an expense existing for a user who has been deleted).

2. DML Operations

Write SQL queries to:

- **Insert 5 users** into the `users` table with unique names and emails.
- **Insert 3 categories** (e.g., 'Food', 'Rent', 'Entertainment').
- **Insert 10 expense records** distributed across different users and categories.
- **Update one incorrect expense:** Use the `UPDATE` command to change the `amount` of a specific `expense_id`.
- **Delete one expense:** Remove a record where the `amount` is less than a specific value.

3. Data Retrieval

Write queries to:

- **Display all expenses with details:** Show the `expense_date`, `amount`, `name` (from users), and `category_name` (from categories) using `INNER JOIN`.
- **Show total expense amount per category:** Use `SUM(amount)` and `GROUP BY category_name`.
- **Display users sorted by total spending:** List user names and their total sum of expenses, ordered from **highest to lowest**.

4. Views

- **Create a view** named `ActiveUsersView`: This view should list the `name` and `email` of any user who has logged **more than 5** individual expense records.
- **Query the view:** Write a simple `SELECT` statement to show all data from `ActiveUsersView`.

Section C: Mini Project

Mini Project: Expense Tracker DB

Using the same schema:

- Write CRUD queries
- Write a stored procedure to calculate **monthly user expense**
- Demonstrate COMMIT and ROLLBACK with example queries

 **Submission:** One .sql file